Wallet Payment Modeling Language

Bianca-Maria Butica
up202603410@up.pt

Mara-Liana Almasan
up2026@up.pt

Hanga Nemeth
up202603316@up.pt

Oana-Maria Silaghi
up202603524@up.pt

I. Introduction

This paper contains a descriptive report of the problem assigned for the first part of the Advanced Software Techniques project. We describe the problem statement, document the process and our understanding of it.

II. Process

A. Problem DSL – Design and Justification

For the Problem DSL, we designed a metamodel that captures the domain of the exact payment problem: given a wallet containing a set of pieces (coins), a payment asks for a target amount to be paid using some of those pieces. We kept the model small and domain-oriented, so that it describes the problem itself, independently of how it will be solved. This is a standard approach in Model-Driven Engineering to separate the problem domain from the solution space [2]. This model can be seen in Figure 1.

The `Wallet` class represents the owner of the pieces and has a single `owner` attribute of type string. The `Piece` class represents an individual coin available in a wallet. It has a `label`, which identifies the piece, and an integer `value`, which is the amount the piece is worth. The `Payment` class represents the request to be solved. It has an integer `amount`, which is the target to be paid, and an integer `maxPieces`, which optionally limits how many pieces may be used.

The classes are connected by two associations. The first one, `wallet_pieces`, links a `Wallet` to its pieces: each `Piece` belongs to exactly one wallet (multiplicity 1), while a wallet may contain any number of pieces (multiplicity $0..*$). The second one, `payment_wallet`, links a `Payment` to the wallet it is paid from, through the role `paidFrom`: each payment is made from exactly one wallet (multiplicity 1), while a wallet may be used in many payments (multiplicity $0..*$). We used regular associations rather than compositions, since the model only needs to express which pieces are available to which payment, and not ownership or lifecycle dependencies. Making the payment refer to the wallet, instead of directly to the pieces, also means that the set of available pieces is defined in a single place and can be reused by several payments.

We also included OCL constraints to ensure that every instance of the model represents a meaningful problem:

- `positiveValue` ensures that every piece is worth more than zero;
- `positiveAmount` ensures that the amount to be paid is greater than zero;
- `amountReachable` ensures that the payment amount does not exceed the total value of all the pieces in the wallet, since otherwise the problem would be trivially unsolvable;
- `positiveMaxPieces` ensures that, if a limit on the number of pieces is given, it is at least one;
- `uniqueLabels` ensures that, within a single wallet, every piece has a different label.

1) Example Object Model: To test our Problem DSL, we made an object model by hand for a concrete payment problem. We took the idea from the example shown in the lecture and adapted it to fit our own metamodel.

a) Problem statement: Ana has a wallet with four coins: 2, 5, 10 and 20 euro cents. She wants to pay exactly 17 euro cents using some of these coins. This is an instance of the Subset Sum problem, a well-known NP-complete problem [4]. Each coin can only be used once, and there is no limit on how many coins she can use. The goal is to find which coins Ana should pick so that their values add up to exactly 17.

b) Model: Ana's wallet is the object `ana_wallet` of class `Wallet`, with `owner` set to "Ana". The four coins are the objects `piece_1`, `piece_2`, `piece_3` and `piece_4` of class `Piece`, with values 2, 5, 10 and 20. Each one also has a label that says what coin it is, for example "2 euro cent coin". All four pieces are connected to `ana_wallet` through the `wallet_pieces` association. The payment is the object `payment` of class `Payment`, with `amount` equal to 17, and it is connected to `ana_wallet` through the `payment_wallet` association. We left `maxPieces` empty because there is no limit on the number of coins.

We also checked that this model respects the OCL constraints we defined: all the pieces have a positive value, the amount is positive, the amount (17) is not bigger than the total value of the wallet (37), and all the labels are different from each other.

Figure 3 shows the resulting object diagram.

B. Solution DSL – Design and Justification

For the Solution DSL, we designed a metamodel that represents linear integer constraint systems, following the mathematical formulation specified for Class C in Annex A of the assignment. The choice of linear constraint is motivated by the fact that they can be solved using Grover's search algorithm, which provides quadratic speedup for this class of problems [3], [1]. This model can be seen in Figure 2. We wanted to create a general model that could represent not only

our exact payment problem, but also other problems involving linear arithmetic constraints.

The `ConstraintSystem` class represents the complete mathematical problem and contains a collection of `IntegerVariable` objects and a root constraint. Each `IntegerVariable` has a name and a `bitWidth` attribute, which determines the range of values it can represent. We used a composition relationship between `ConstraintSystem` and `IntegerVariable` because the variables belong to the constraint system.

To represent mathematical expressions, we created the `LinearExpression` and `LinearTerm` classes. A `LinearExpression` contains a constant and a collection of terms, while each `LinearTerm` has a coefficient and references an `IntegerVariable`. We used composition between `LinearExpression` and `LinearTerm` because the terms are part of the expression. However, we used a regular association between `LinearTerm` and `IntegerVariable`, since the same variable may appear in multiple mathematical expressions.

We introduced an abstract class called `ConstraintNode` to provide a common structure for different types of constraints. The `AtomicConstraint` class represents a comparison between a linear expression and zero, using either `EQ` (equal) or `LE` (less than or equal). The `LogicalConstraint` class allows multiple constraints to be combined using `AND` or `OR`. Both classes inherit from `ConstraintNode`, allowing us to represent simple constraints as well as more complex combinations. These operators were chosen because they correspond to the intermediate mathematical model defined in Annex A, Class C.

We also included OCL constraints to improve the correctness of our models. For example, `PositiveBitWidth` ensures that variables use at least one bit, `UniqueVariableNames` prevents duplicate variable names within a system, and `NonZeroCoefficient` prevents unnecessary terms with zero coefficients. The `NoSelfReference` rule prevents a logical constraint from directly referencing itself.

1) Example Solution Model: To demonstrate our Solution DSL, we manually created an object model corresponding to an exact payment problem. This object model can be seen in figure 4. The available coin values are 2, 5, 10 and 20, and the target amount is 17.

We represented each coin using an `IntegerVariable` with `bitWidth = 1`, meaning that each coin can either be selected or not selected. The corresponding `LinearTerm` objects have coefficients 2, 5, 10 and 20, while the `LinearExpression` has a constant of -17. Finally, an `AtomicConstraint` with the operator `EQ` ensures that the expression equals zero.

The resulting mathematical formulation is:

$$2x_1 + 5x_2 + 10x_3 + 20x_4 - 17 = 0 \tag{1}$$

One valid solution is:

$$(x_1, x_2, x_3, x_4) = (1, 1, 1, 0) \tag{2}$$

This corresponds to selecting the coins with values 2, 5 and 10.

This example demonstrates how our Solution DSL can represent a concrete problem independently of its original domain, providing the intermediate mathematical representation needed for the model-to-model transformations in Part II.

III. Concept Table

A. Problem model

Concept	Intrinsic Properties	Extrinsic Properties
WalletPaymentModel	name : String	One *Wallet* One *Payment*
Wallet	name : String	Arbitrary number of *Pieces*
Piece	id : String value : Integer	
Payment	targetAmount : Integer maxPieces : Integer	Arbitrary number of selected *Pieces*

TABLE I
WPML Concept List

B. Solution model

Concept	Intrinsic Properties	Extrinsic Properties
ConstraintSystem	name : String	One root *ConstraintNode* Arbitrary number of *IntegerVariables*
ConstraintNode		
LogicalConstraint	operator : [AND—OR]	Arbitrary number of operand *ConstraintNodes*
AtomicConstraint	operator : [EQ—LE]	One *LinearExpression*
LinearExpression	constant : Integer	Arbitrary number of *LinearTerms*
LinearTerm	coefficient : Integer	One referenced *IntegerVariable*
IntegerVariable	name : String bitWidth : Integer	

TABLE II
Solution DSL Concept List.

IV. Diagrams

A. Problem model

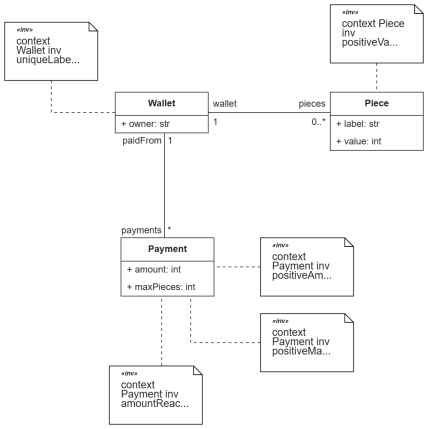


Fig. 1. Problem DSL metamodel

B. Solution model

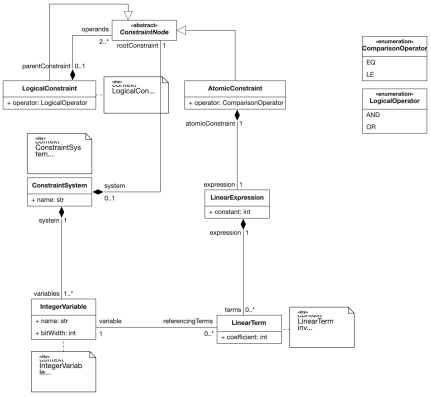


Fig. 2. Solution DSL metamodel

C. Examples

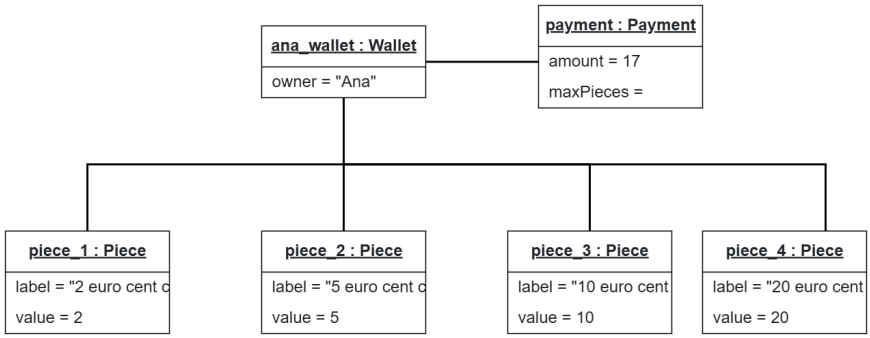


Fig. 3. Object diagram of the example problem model (Ana's wallet).

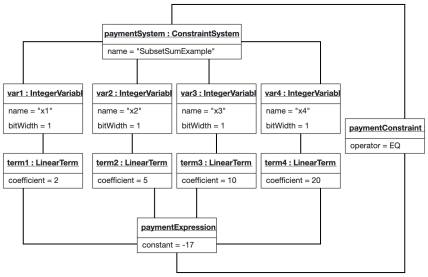


Fig. 4. Object diagram of the example solution model

References

[1] Michel Boyer, Gilles Brassard, Peter Høyer, and Alain Tapp. Tight bounds on quantum searching. *Fortschritte der Physik*, 46(4-5):493–505, 1998.

[2] Felix Gemeinhardt, Antonio Garmendia, and Manuel Wimmer. Towards model-driven quantum software engineering. pages 13–15, 06 2021.

[3] Lov K. Grover. A fast quantum mechanical algorithm for database search, 1996.

[4] Richard M. Karp. *Reducibility among Combinatorial Problems*, pages 85–103. Springer US, Boston, MA, 1972.